MODULE 1 · LECTURE 4

Operating Systems: Definition, Types, Functions and Applications

What an operating system does, how a computer starts, how programs share the processor, memory and files, the main types of operating system, and where each is used in agriculture.

Learning objectives

After this lecture you should be able to:

1. Define an operating system (OS) and explain its roles.
2. Describe the booting process and the structure of an OS (kernel and shell).
3. Explain process management, including process states and CPU scheduling.
4. Explain memory, file and device management and OS security.
5. Classify operating systems and give examples, including real-time and mobile systems.
6. Choose a suitable operating system for agricultural uses.

Scope: Lecture 3 described memory and storage; the OS is the program that manages them. Lecture 5 covers the MS Office applications that run on the OS, and Lecture 7 the Internet.

1. What is an operating system?

Definition. An operating system is the system software that manages a computer's hardware and software resources and provides common services to application programs, acting as the interface between the user and the hardware (Silberschatz et al., 2018).

Without an OS, every program would have to control the keyboard, screen, disk and memory on its own. The OS does this once, for all programs. It has three roles:

- **Interface (extended machine)** — it hides the details of the hardware and gives users and programs a simple way to work, through menus, icons, commands and files.
- **Resource manager** — it shares the processor, memory, storage and devices among programs and users fairly and efficiently.
- **Control program** — it prevents errors and misuse, so that one program cannot damage another or the system.

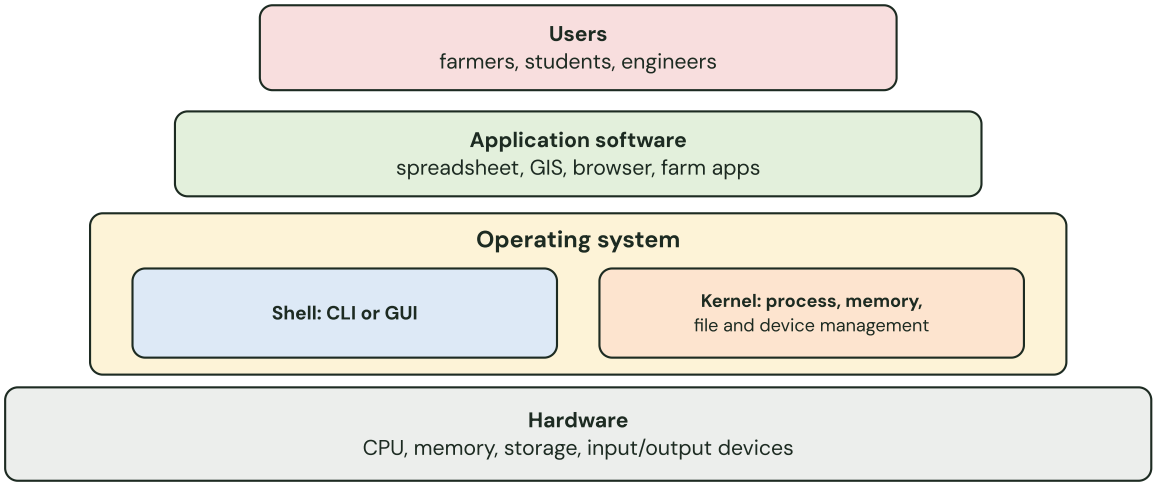


Figure 1. The operating system sits between application software and the hardware. Original diagram.

2. Booting: how a computer starts

Booting is the process of loading the operating system into memory when the computer is switched on.

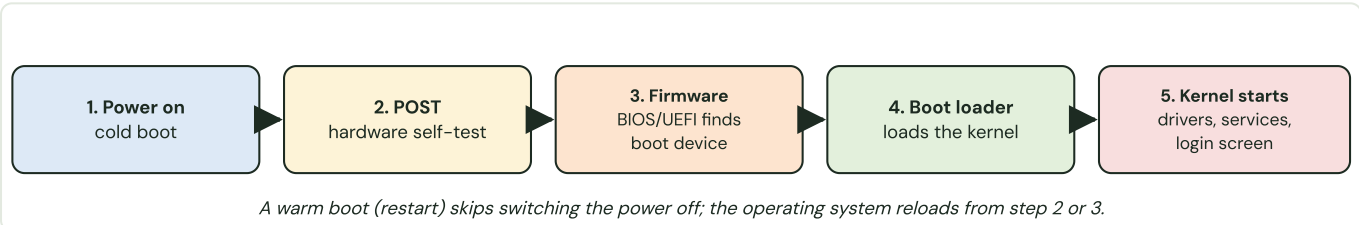


Figure 2. Steps in booting a computer. Original diagram.

1. **Power on** — the processor starts executing the firmware stored in ROM/flash (Lecture 3).
2. **POST (power-on self-test)** — the firmware checks that the processor, memory, keyboard and drives work.
3. **Firmware (BIOS or UEFI)** — finds the boot device (SSD, hard disk or USB drive).
4. **Boot loader** — a small program on the boot device loads the OS kernel into RAM.
5. **Kernel start-up** — the kernel loads drivers, starts system services and shows the login screen.

A **cold boot** starts from power-off; a **warm boot** (restart) reloads the OS without switching the power off.

3. Structure of an operating system

Part	Role
Kernel	The core of the OS, always in memory. It manages processes, memory, files and devices and runs with full control of the hardware
Shell	The user interface to the kernel — either a **command-line interface (CLI)** or a **graphical user interface (GUI)**
System calls	The defined requests a program makes to the kernel, such as "open a file" or "send data to the printer"
Device drivers	Programs that let the kernel control particular devices (Lecture 2)
System utilities	Programs for tasks such as formatting disks, managing files and checking for errors

Kernels are designed in different ways. A **monolithic** kernel (Linux) runs all core services in one large program; a **microkernel** keeps only the essentials in the kernel and runs other services separately for reliability; **hybrid** kernels (Windows, macOS) mix the two approaches.

4. Functions of an operating system

4.1 Process management

A **program** is a set of instructions stored on disk; a **process** is a program in execution, with its own memory and state. The OS creates and ends processes, shares the CPU among them, and lets them communicate. As a process runs, it moves between five states:

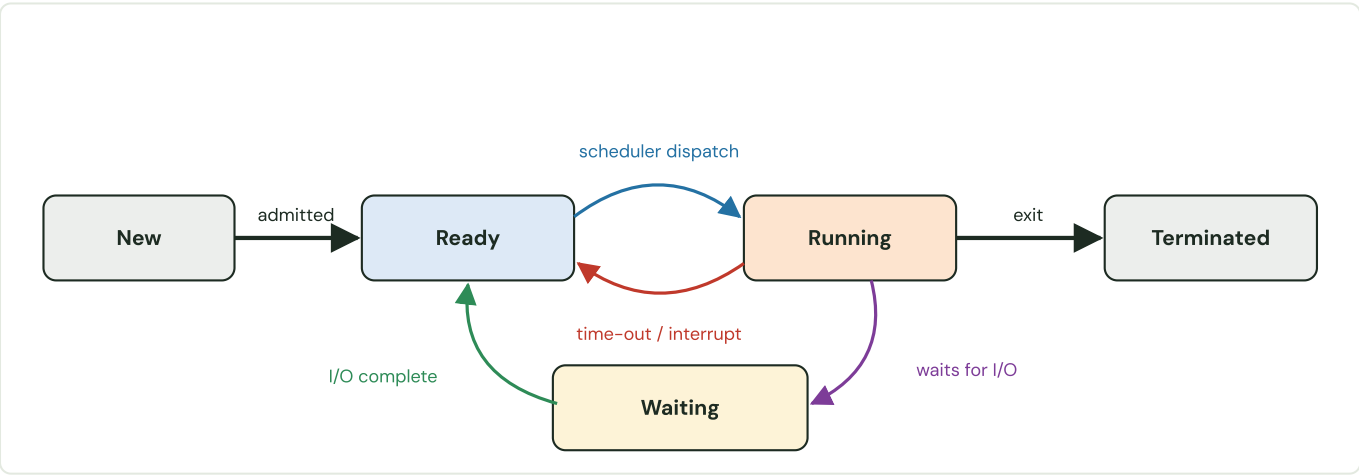


Figure 3. Process states and transitions. Original diagram.

Many processes may be **ready** at once, but one CPU core runs only one at a time. The **CPU scheduler** decides which runs next. Switching the CPU from one process to another is a **context switch**. Common scheduling methods are:

Method	Rule	Strength / weakness
First-come, first-served (FCFS)	Run processes in order of arrival	Simple; short jobs may wait behind a long one
Shortest job first (SJF)	Run the process with the shortest CPU time next	Lowest average waiting time; needs to know run times
Priority	Run the highest-priority process	Important tasks first; low-priority tasks may starve
Round robin (RR)	Each process gets a fixed time slice (quantum) in turn	Fair and responsive; used in time-sharing systems

Worked example — CPU scheduling
Three processes arrive together with CPU times P1 = 8 ms, P2 = 4 ms and P3 = 2 ms. Waiting time = time spent in the ready queue.
FCFS (P1, P2, P3): waits 0, 8, 12 → average = 20 ÷ 3 = 6.67 ms.
SJF (P3, P2, P1): waits 0, 2, 6 → average = 8 ÷ 3 = 2.67 ms.
Round robin, quantum 2 ms: P1 finishes at 14, P2 at 10, P3 at 6. Waiting = finish − CPU time: P1 = 6, P2 = 6, P3 = 4 → average = 16 ÷ 3 = 5.33 ms.
SJF gives the lowest average wait; round robin gives every process a share of the CPU quickly.

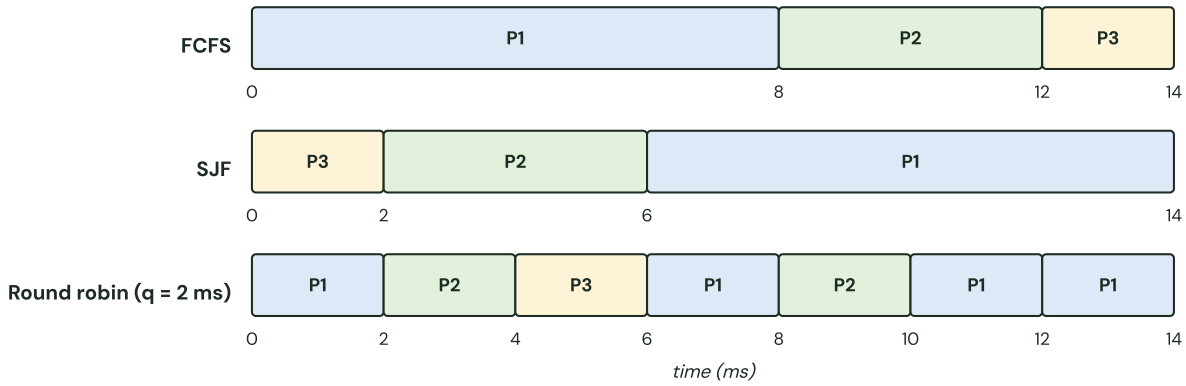


Figure 4. Gantt charts for the worked example. Original diagram.

4.2 Memory management

- The OS keeps track of which parts of RAM are in use, allocates memory to processes, frees it when they finish, and **protects** each process's memory from the others.
- With **paging**, memory is divided into fixed-size blocks (often 4 KB). A process's pages need not sit side by side in RAM, and pages not in use can be kept on disk — **virtual memory** (Lecture 3).

Worked example — paging
Page size = 4 KB = 4096 bytes. Where is logical address 10,000?
Page number = 10,000 ÷ 4096 = 2 (whole number part); offset = 10,000 − 2 × 4096 = 1808.
So the address is byte 1808 of page 2. The OS's page table tells the hardware which RAM frame holds page 2.

4.3 File management

- A **file** is a named collection of related data stored on disk; a **folder (directory)** groups files in a tree. The OS creates, names, copies, moves, deletes and protects files.

- A **path** gives a file's location — e.g. C:\Farm\2026\rainfall.csv in Windows or /home/student/farm/rainfall.csv in Linux.
- The **file extension** shows the type: .docx (Word), .xlsx (Excel), .csv (comma-separated data from loggers), .jpg (images), .pdf, .shp (GIS shapefile).

File system	Used by	Note
FAT32	Older memory cards and pen drives (SDHC cards)	No single file can exceed 4 GB (2^{32} - 1 bytes) — long drone or field videos may not copy
exFAT	SDXC/SDUC cards, large pen drives	Removes the 4 GB file limit (Microsoft, n.d.); widely supported by computers and cameras
NTFS	Windows system drives	Permissions, encryption and journaling for reliability
ext4	Linux	Common Linux file system
APFS	macOS and iOS	Apple's file system for flash storage

4.4 Device management

- The OS controls devices through **drivers**, responds to **interrupts** (signals from devices that need attention), and uses **buffers** to smooth out speed differences between devices.
- **Spooling** places jobs for a slow device, such as a printer, in a queue on disk so that programs need not wait.

4.5 Security and user management

- **User accounts and authentication** — passwords, PINs or fingerprints. Never share passwords, and use a separate account for each user.
- **Permissions** — each file has read, write and execute rights for different users, so farm accounts can be read by staff but changed only by the owner.
- **Updates and patches** fix security flaws; a **firewall** and **anti-malware** software protect against attacks.

4.6 Other functions

- **Networking** — connecting to Wi-Fi, mobile data and shared printers (Lecture 7).
- **User interface** — the CLI or GUI through which people work.
- **Error detection and logging** — detecting hardware and software faults and recording events for troubleshooting.
- **Accounting** — tracking the use of resources, for example in shared university servers.

5. Types of operating systems

Type	Main idea	Example or use
Batch	Similar jobs are collected and run together without user interaction	Early mainframes; overnight processing of large survey or payroll data
Multiprogramming	Several programs are kept in memory so the CPU always has work while others wait for I/O	Basis of all modern OS
Time-sharing / multitasking	The CPU switches rapidly among tasks or users, so each seems to have the computer to itself	Desktop OS running a browser, spreadsheet and music together
Multiprocessing	Uses two or more CPUs or cores together	Servers and modern multi-core PCs
Multi-user	Many users share one computer at the same time	University and research servers running Linux
Real-time (RTOS)	Responds within a guaranteed time. **Hard** real-time: a missed deadline is a failure; **soft** real-time: occasional delays are tolerable	Drone flight controllers (the PX4 autopilot runs mainly on the NuttX RTOS — PX4 Autopilot, n.d.), engine and brake control, robotic milking
Distributed	Many networked computers appear as one system	Cloud services and large data analysis
Network OS	Manages shared files, printers and users over a network	Office servers in an agricultural university
Mobile	Designed for touch screens, batteries and wireless links	Android, iOS
Embedded	Small OS built into a device for one purpose	Irrigation controllers, tractor terminals, weather-station loggers

6. Popular operating systems

Year	Milestone
1969	UNIX begun at Bell Labs by Ken Thompson and Dennis Ritchie; rewritten in C in 1972 (Nokia Bell Labs, n.d.)
1981	MS-DOS, a command-line OS, ships with the IBM PC
1985	Microsoft Windows 1.0 adds a graphical interface on top of MS-DOS
1991	Linus Torvalds announces the Linux kernel on 25 August 1991 (Torvalds, 1991)
2007–2008	Apple's iOS (2007) and Google's Android (2008, built on the Linux kernel) launch the smartphone era

OS	Developer and licence	Where it is used
Windows	Microsoft; proprietary	Most office and student PCs
macOS	Apple; proprietary	Apple computers
Linux (Ubuntu, Debian and others)	Community; open-source	Servers, research computing, single-board computers for farm IoT projects
BOSS Linux	C-DAC Chennai; open-source, Debian-based, with Indian-language support	Government and educational use in India (Linux Magazine, 2024)
Android	Google; open-source core with proprietary services	About 94% of mobile phones in India in September 2026 (StatCounter, 2026) — so most farm apps are built for Android (Lecture 14)
iOS	Apple; proprietary	iPhones
RTOS (NuttX, FreeRTOS and others)	Mostly open-source	Drones, controllers and other embedded devices

Market-share figures change month to month; check the source for the latest values before quoting them.

7. Command-line and graphical interfaces

	Command-line interface (CLI)	Graphical user interface (GUI)
How you work	Type text commands	Point and click on windows, icons and menus
Ease of learning	Harder; commands must be remembered	Easier for beginners
Speed and automation	Fast for experts; commands can be scripted to repeat jobs	Slower for repetitive tasks
Resources	Needs little memory and processing	Needs more memory and graphics power
Example	Windows Command Prompt, Linux terminal	Windows desktop, Android home screen

Task	Windows command	Linux command
List files in a folder	dir	ls
Change folder	cd Farm	cd farm
Show current folder	cd	pwd
Make a folder	mkdir 2026	mkdir 2026
Copy a file	copy rain.csv backup.csv	cp rain.csv backup.csv
Move or rename	move a.csv b.csv	mv a.csv b.csv
Delete a file	del old.csv	rm old.csv
Clear the screen	cls	clear

8. Operating systems in agriculture

Setting	Typical OS	Why
Farmer's smartphone	Android	Low-cost phones; most advisory, market and weather apps
Agricultural office and student PCs	Windows or Linux	Office software, GIS (QGIS runs on Windows, Linux and macOS), statistics
Research servers and data portals	Linux	Stable, multi-user, free to run on many servers
Low-cost farm IoT gateways	Linux on single-board computers	Collect sensor data and send it to the cloud (Lecture 23)
Drones	RTOS on the flight controller	Stable flight needs guaranteed response times
Tractor terminals and implement controllers	Embedded OS or RTOS	Reliable control of hydraulics and seeding rates (Lecture 12)
Irrigation and greenhouse controllers	Embedded firmware or RTOS	Run the same control program for years without attention

Choosing and maintaining an OS

- **Software needed** — check that the GIS, statistics or design programs you need run on the OS.
- **Cost and licence** — open-source systems such as Linux and BOSS save licence fees; proprietary systems may come with the computer.
- **Hardware** — older or low-cost machines run lighter Linux versions better.
- **Language** — Indian-language support matters for farmer-facing computers and kiosks.
- **Security** — install updates, use strong passwords, back up data (Lecture 3) and avoid pirated software, which often carries malware.

9. Key takeaways

- An operating system is the interface between users and hardware, the manager of resources and the guard against misuse.
- Booting loads the OS: power on → POST → firmware → boot loader → kernel.
- The kernel manages processes, memory, files and devices; the shell (CLI or GUI) is how people use it.
- Processes move through new, ready, running, waiting and terminated states; schedulers such as FCFS, SJF and round robin share the CPU.
- Memory management uses paging and virtual memory; file systems such as FAT32, exFAT, NTFS and ext4 organise storage.
- OS types include batch, multitasking, multiprocessing, multi-user, real-time, distributed, network, mobile and embedded.
- In agriculture, Android dominates farmers' phones, Linux runs servers and IoT gateways, and real-time systems control drones and machines.

10. Examination questions

Short-answer questions (2 marks)

Q1. Define an operating system.

System software that manages hardware and software resources, provides services to programs and acts as the interface between the user and the hardware.

Q2. What is booting? Distinguish cold and warm boot.

Loading the OS into memory at start-up. A cold boot starts from power-off; a warm boot restarts without switching off.

Q3. What is POST?

Power-on self-test — the firmware's check of the hardware when the computer is switched on.

Q4. Distinguish between the kernel and the shell.

The kernel is the core that manages resources; the shell is the user interface (CLI or GUI) to the kernel.

Q5. Distinguish between a program and a process.

A program is a stored set of instructions; a process is a program in execution.

Q6. Name the five process states.

New, ready, running, waiting and terminated.

Q7. What is round-robin scheduling?

Each ready process receives a fixed time slice (quantum) in turn.

Q8. What is spooling?

Queuing jobs for a slow device, such as a printer, on disk so programs need not wait for it.

Q9. What is a real-time OS? Give an agricultural example.

An OS that responds within a guaranteed time — e.g. the RTOS in a drone's flight controller.

Q10. Why can a FAT32 memory card not store a 6 GB video file?

FAT32 limits a single file to 4 GB (2^{32} - 1 bytes); exFAT removes the limit.

Long-answer questions (5–10 marks) — answer outlines

Q1. Explain the functions of an operating system.

Process, memory, file and device management; security; networking; user interface; error detection and accounting — with examples.

Q2. Describe the booting process with a diagram.

Figure 2: power on, POST, BIOS/UEFI, boot loader, kernel; cold and warm boot.

Q3. Explain process states and CPU scheduling with an example.

Figure 3; FCFS, SJF, priority and round robin; the worked example and Gantt charts (Figure 4).

Q4. Classify operating systems with examples.

Batch, multiprogramming, time-sharing, multiprocessing, multi-user, real-time (hard/soft), distributed, network, mobile and embedded (table in Section 5).

Q5. Compare CLI and GUI, and list common commands.

Comparison table and the Windows/Linux command table in Section 7.

Q6. Discuss the role of operating systems in agriculture.

Android on farmers' phones, Windows/Linux in offices, Linux servers and IoT gateways, RTOS in drones and machines; how to choose and maintain an OS.

11. Glossary

Term	Meaning
Booting	Loading the operating system at start-up
CLI	Command-line interface — typed commands
Context switch	Saving one process's state and loading another's
Device driver	Software that lets the OS control a device
File system	The method an OS uses to organise files on storage (FAT32, exFAT, NTFS, ext4)
GUI	Graphical user interface — windows, icons, menus and pointer
Interrupt	A signal from hardware or software asking for the CPU's attention
Kernel	The core of the OS that manages resources
Multitasking	Running several tasks by switching the CPU among them
Open-source	Software whose source code may be studied, changed and shared
Paging	Dividing memory into fixed-size blocks called pages
POST	Power-on self-test
Process	A program in execution
Quantum	The time slice given to each process in round-robin scheduling
RTOS	Real-time operating system — guarantees response times
Scheduler	The part of the OS that decides which process runs next
Shell	The user interface to the kernel
Spooling	Queuing jobs for a slow device on disk
System call	A request from a program to the kernel
Virtual memory	Using disk space as an extension of RAM

References

Linux Magazine. (2024). Who's the BOSS? India's national distribution. *Linux Magazine*, (288). https://www.linux-magazine.com/Issues/2024/288/Who-s-the-BOSS

Microsoft. (n.d.). *exFAT file system specification*. Microsoft Learn. Retrieved 8 November 2026, from https://learn.microsoft.com/en-us/windows/win32/fileio/exfat-specification

Nokia Bell Labs. (n.d.). *50 years of Unix*. Retrieved 8 November 2026, from https://www.nokia.com/bell-labs/about/history/innovation-stories/50-years-unix/

PX4 Autopilot. (n.d.). *PX4 architectural overview*. PX4 User Guide (Dronecode Foundation). Retrieved 8 November 2026, from https://docs.px4.io/main/en/concept/architecture.html

Rajaraman, V., & Adabala, N. (2015). *Fundamentals of computers* (6th ed.). PHI Learning. ISBN 978-81-203-5067-0

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley. ISBN 978-1-119-32091-3

Sinha, P. K., & Sinha, P. (2004). *Computer fundamentals* (6th ed.). BPB Publications. ISBN 978-81-7656-752-7

StatCounter. (2026). *Mobile operating system market share India* [Data set]. StatCounter Global Stats. Retrieved 8 November 2026, from https://gs.statcounter.com/os-market-share/mobile/india

Torvalds, L. (1991, August 25). *What would you like to see most in minix?* [Online forum post]. Usenet newsgroup comp.os.minix.

All diagrams are original. Explanations are written in original language; standard content on OS structure, process states, scheduling and memory management follows Silberschatz, Galvin and Gagne (2018). The scheduling and paging worked examples were prepared for this lecture.

Practice MCQs

Q1. The operating system acts as an interface between

(a) two computers
(b) the user and the hardware
(c) RAM and ROM
(d) the CPU and the ALU

Q2. POST stands for

(a) Power-on self-test
(b) Primary operating system test
(c) Program output status table
(d) Power output system test

Q3. The core of the operating system that manages resources is the

(a) shell
(b) kernel
(c) driver
(d) boot loader

Q4. A program in execution is called a

(a) file
(b) thread
(c) process
(d) driver

Q5. Which scheduling method gives the lowest average waiting time when run times are known?

(a) FCFS
(b) SJF
(c) Round robin
(d) Priority

Q6. Processes with CPU times 8, 4 and 2 ms are run FCFS in that order. Average waiting time is

(a) 2.67 ms
(b) 5.33 ms
(c) 6.67 ms
(d) 8 ms

Q7. With 4 KB pages, logical address 10,000 lies in page

(a) 1
(b) 2
(c) 3
(d) 4

Q8. Queuing print jobs on disk is called

(a) paging
(b) spooling
(c) swapping
(d) booting

Q9. A drone flight controller usually runs a(n)

(a) batch OS
(b) real-time OS
(c) network OS
(d) multi-user OS

Q10. BOSS Linux was developed by

(a) ISRO
(b) C-DAC
(c) IIT Bombay
(d) NIC

Q11. The largest single file on a FAT32 volume is about

(a) 2 GB
(b) 4 GB
(c) 32 GB
(d) 2 TB

Q12. The Linux command to list files is

(a) dir
(b) ls
(c) cd
(d) pwd

Answers

Q1 — (b) the user and the hardware

Q2 — (a) Power-on self-test
Q3 — (b) kernel
Q4 — (c) process
Q5 — (b) SJF
Q6 — (c) 6.67 ms — *(0 + 8 + 12) ÷ 3.*
Q7 — (b) 2 — *10,000 ÷ 4096 = 2.44.*
Q8 — (b) spooling
Q9 — (b) real-time OS
Q10 — (b) C-DAC
Q11 — (b) 4 GB
Q12 — (b) ls